COMP 418/518: Homework 4

[Weight of homework: 25% of the final grade]

Authors (fill in your names here)

Due on April 6, 2026
(released on March 23, 2026)

Instructions: For this homework, you should work in groups of 3–4 people (i.e., the groups that have been declared and accepted). Only one member of the group should submit on Gradescope. All group members must be included in the submission on the Gradescope system (otherwise no credit will be given). Also specify in your submission file all the members of your group (name and NetID) using the `\authors` macro. The submission must be typed in LaTeX using the provided template. The use of AI systems (including, but not limited to, ChatGPT, Gemini, Copilot, and Claude) is entirely allowed. No discussion or collaboration is allowed outside of the group. Late submissions are not accepted.

You should provide *pseudocode* for all the algorithms that you describe.

Grading: We will give a 5% grade bonus to students taking COMP 418. The homework will be graded based on correctness, completeness, and the quality of the provided solutions.

Note: Please go over the provided Java source code, because it contains some definitions that are relevant for several questions of the homework. Unless specified otherwise, all the denotations that we consider in the problems are cumulative (i.e., they give the overall output history).

1 The Dataflow Model [110 points]

We will explore the dataflow model of computation. We consider dataflow graphs where each process (sometimes referred to as node, operator, actor) can be implemented using the following Java interface:

```
public interface Operator<A,B> {
        void start(Sink<B> sink);
        void next(A item, Sink<B> sink);
}

public interface Sink<A> {
        void next(A item);
}
```

See the uploaded code for a similar programming model.

We assume that every execution of `start` and `next` terminates and thus results in the generation of a *finite* number of output items. Under this assumption, every operator can be described by a *monotone* function $f : A^* \to B^*$ (cumulative denotation), where A^* is the set of all finite words (strings) over A. See several lectures that discuss this concept of describing operators (streaming algorithms) using functions.

1.1 Semantic Characterization of Statelessness [10 points]

You are asked to give a semantic characterization of the *stateless* operators, that is, those operators that can be implemented without maintaining any state across steps (examples of such operators: `id`, `filter`, and `map`). The operators `fold` and `scan`, on the other hand, are stateful. More specifically, you want to characterize the class of (computable) monotone functions $f : A^* \to B^*$ that admit a stateless implementation. Formulate precisely a property $P(f)$ so that Theorem 1 below holds.

Definition of property $P(f)$: There exists a function $h : A \to B^*$ such that for all $w \in A^*$ and all $a \in A$:

$$f(w \cdot a) = f(w) \cdot h(a)$$

In words, the incremental output produced by each new input element a is independent of the prior input history w. Equivalently, if we define $\Delta_f(w, a)$ to be the unique string $\delta \in B^*$ such that $f(w \cdot a) = f(w) \cdot \delta$ (which exists because f is monotone), then $\Delta_f(u, a) = \Delta_f(v, a)$ for all $u, v \in A^*$ and $a \in A$.

Theorem 1. Let $f : A^* \to B^*$ be a computable monotone function. The function f is the cumulative denotation of a stateless operator if and only if $P(f)$.

Proof. ($\Rightarrow$) Suppose f is the cumulative denotation of a stateless operator. Since the operator is stateless, `start(sink)` produces a fixed output $s_0 \in B^*$ (with no dependence on any mutable state), and for each $a \in A$, `next(a, sink)` produces output that depends only on a, which we denote $h(a) \in B^*$. The cumulative denotation satisfies $f(\varepsilon) = s_0$, and for all $w \in A^*$ and $a \in A$:

$$f(w \cdot a) = f(w) \cdot h(a).$$

This holds because processing each new element a appends exactly $h(a)$ to the cumulative output, regardless of the prior input history w (since no state is carried across invocations of `next`). Hence $P(f)$ holds.

($\Leftarrow$) Suppose $P(f)$ holds, i.e., there exists $h : A \to B^*$ such that $f(w \cdot a) = f(w) \cdot h(a)$ for all $w \in A^*$ and $a \in A$. We construct a stateless operator as follows:

- `start(sink)`: emit each element of $f(\varepsilon)$ to the sink.
- `next(a, sink)`: emit each element of $h(a)$ to the sink.

This operator is stateless: neither method accesses or modifies any mutable state that depends on previous inputs. The function h is computable because $h(a) = f(\varepsilon)^{-1} \cdot f(a)$ (removing the prefix $f(\varepsilon)$ from $f(a)$, which is well-defined by monotonicity of f).

Let f' denote the cumulative denotation of this operator. We show $f' = f$ by induction on $|w|$.

Base case: $f'(\varepsilon) = f(\varepsilon)$ by construction of `start`.

Inductive step: Assume $f'(w) = f(w)$ for some $w \in A^*$. For any $a \in A$:

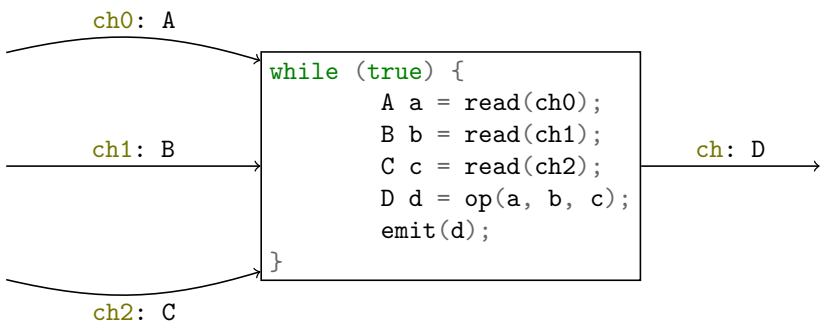
$$f'(w \cdot a) = f'(w) \cdot h(a) = f(w) \cdot h(a) = f(w \cdot a),$$

where the first equality follows from the definition of the operator, the second from the inductive hypothesis, and the third from $P(f)$.

By induction, $f'(w) = f(w)$ for all $w \in A^*$. Therefore f is the cumulative denotation of a stateless operator. $\square$

1.2 Translations Between Programming Models [25 points]

We have used extensively a notation to describe dataflow processes using `read(channel)` to read from an input channel and `emit(channel)` to write to an output channel. This programming model was introduced by Kahn in [1] and was further elaborated in [2]. If you want to learn more about these seminal works you can read the survey [4] and the notes [3] (see reading list on Canvas). Consider the following Kahn process:



Describe the same computation by implementing the `Zip3` operator of type `Operator<Or3<A,B,C>,D>`. The three input channels of the Kahn process are encoded using the disjoint union (coproduct) `Or3<A,B,C>`, which is a straightforward extension of `Or<A,B>` (see the uploaded code). Fill pseudocode in Figure 1. You do not have to write actual Java code for this.

Consider now the operator `Count2` of type `Operator<Or<A,A>,Nat>` that is shown in Figure 2. The input can be thought of as the interleaving of two input channels. Translate `Count2` into a Kahn process with two input channels (if possible), or prove that it is not possible to do so.

Operator to Kahn process translation: **Claim:** It is *not possible* to translate `Count2` into a Kahn process with two input channels.

Proof. Suppose for contradiction that there exists a Kahn process P with input channels `ch0: A` and `ch1: A` and output channel `ch: Nat` whose cumulative denotation matches that of `Count2`. That is, the output function satisfies $g(x, y) = 1, 2, \ldots, |x| + |y|$ for all input histories x on `ch0` and y on `ch1`.

A Kahn process is a sequential program whose only input operation is a blocking `read(ch)` from a specific channel. At the very start of execution (both channels empty), P must attempt to `read` from either `ch0` or `ch1` in order to make progress and eventually produce output.

Case 1: Suppose P first executes `read(ch0)`. Consider the input $x = \varepsilon$ (no items on `ch0`) and $y = b_1$ (one item on `ch1`). Since P attempts to read from `ch0` first and `ch0` is permanently empty, P blocks indefinitely and produces no output. The output of P is ε. However, $g(\varepsilon, b_1) = [1] \neq \varepsilon$, a contradiction.

Case 2: Suppose P first executes `read(ch1)`. Then choosing $x = a_1, y = \varepsilon$ yields $g(a_1, \varepsilon) = [1] \neq \varepsilon$, the same contradiction.

Since these are the only two possible first operations, no Kahn process can compute the function of `Count2`.

The fundamental issue is that `Count2` responds to items arriving on *either* channel, which requires a non-deterministic merge (or `select`) operation. The Kahn process model only supports deterministic, blocking reads from a single channel, and therefore cannot express this behavior. □

```java
public class Zip3<A, B, C, D> implements Operator<Or3<A,B,C>, D> {

        private final Func3<A,B,C,D> op;
        private Queue<A> bufA;
        private Queue<B> bufB;
        private Queue<C> bufC;

        public Zip3(Func3<A,B,C,D> op) {
                this.op = op;
                bufA = new LinkedList<>();
                bufB = new LinkedList<>();
                bufC = new LinkedList<>();
        }

        @Override
        public void start(Sink<D> sink) {
                // no output on start
        }

        @Override
        public void next(Or3<A,B,C> item, Sink<D> sink) {
                if (item.isLeft) {
                        bufA.add(item.getLeft());
                } else if (item.isMid) {
                        bufB.add(item.getMid());
                } else {
                        bufC.add(item.getRight());
                }
                while (!bufA.isEmpty() && !bufB.isEmpty()
                       && !bufC.isEmpty()) {
                        D d = op.apply(bufA.poll(),
                                       bufB.poll(), bufC.poll());
                        sink.next(d);
                }
        }
}
```

Figure 1: The `Zip3` operator.

```java
public class Count2 implements Operator<Or<A,A>, Nat> {

        private long count = 0;

        @Override
        public void start(Sink<Nat> sink) { /* do nothing */  }

        @Override
        public void next(Or<A,A> item, Sink<Nat> sink) {
                A a = item.map(x -> x, x -> x);
                count += 1;
                sink.next(count);
        }
}
```

Figure 2: The `Count2` operator.

1.3 Denotations [10 points]

Describe the denotation of `Zip3` as a function $f : A^* \times B^* \times C^* \to D^*$ and the denotation of `Count2` as a function $g : A^* \times A^* \to \mathbb{N}^*$, where $\mathbb{N}$ is the set of the natural numbers (nonnegative integers).

Solution: Denotation of Zip3. The function $f : A^* \times B^* \times C^* \to D^*$ is defined by:

$$f(a_1 \cdots a_m,\ b_1 \cdots b_n,\ c_1 \cdots c_p) \ = \ d_1\, d_2 \cdots d_k$$

where $k = \min(m, n, p)$ and $d_i = op(a_i, b_i, c_i)$ for $i = 1, \ldots, k$. That is, `Zip3` outputs one element for each complete triple of inputs, processing them in order. If any of the three input channels has fewer items than the others, the excess items on the longer channels are not yet processed (the operator waits for matching items that have not yet arrived).

This function is monotone: if $(x, y, z) \sqsubseteq (x', y', z')$ (componentwise prefix order), then $\min(|x|, |y|, |z|) \leq \min(|x'|, |y'|, |z'|)$ and the corresponding output triples agree on the shared prefix, so $f(x, y, z) \sqsubseteq f(x', y', z')$.

Denotation of Count2. The function $g : A^* \times A^* \to \mathbb{N}^*$ is defined by:

$$g(a_1 \cdots a_m,\ b_1 \cdots b_n) \ = \ 1,\ 2,\ 3,\ \ldots,\ m + n$$

That is, the output is the sequence of natural numbers from 1 to $|x| + |y|$, where $|x| = m$ and $|y| = n$ are the lengths of the two input sequences. `Count2` ignores the actual values and channel tags of the input items and simply counts the total number of items received.

This function is monotone: if $(x, y) \sqsubseteq (x', y')$, then $|x| + |y| \leq |x'| + |y'|$, so $g(x, y) \sqsubseteq g(x', y')$.

Remark: Although g is a well-defined monotone function, the *timing* of when each count value is emitted depends on the interleaving of the two input channels. This is precisely why `Count2` cannot be implemented as a Kahn process, as shown in Section 1.2.

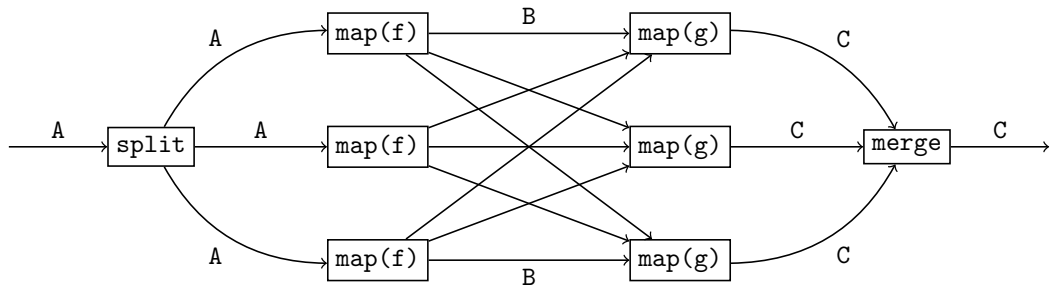
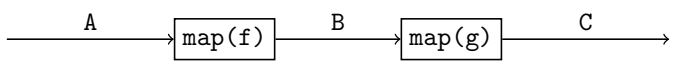


Figure 3: Extracting data parallelism from a simple pipeline of stateless operators.

1.4 Data Parallelism [25 points]

In applications that require the processing of massive amounts of data (e.g., data generated by a large number of high-resolution sensors) it is often desirable to exploit *data parallelism*. This is what the distributed data processing systems MapReduce, Spark, Storm and others do. As a simple example of this, consider a pipeline consisting of two `map` operators:



We wish to parallelize this pipeline by creating several instances of `map(f)` and `map(g)` that compute independently. This is a meaningful transformation when `f` and `g` are computationally expensive. This transformation gives rise to dataflow graph of Figure 3.

Describe each of the nodes (processes) of the dataflow graph of Figure 3. You can assume that every communication channel is reliable (no messages are lost) and behaves like a FIFO buffer. Make sure that the behavior of the graph of Figure 3 (in terms of the input/output function) is exactly the same as the behavior of the original pipeline. Does this require making any noteworthy changes to the graph? You may have to introduce new notation for writing pseudocode that describes the nodes. Try to make your notation intuitive and explain the aspects that are not obvious. You do not have to restrict yourself to Kahn processes for the code that runs on each node.

Solution: We describe each node of the dataflow graph in Figure 3. Let $N = 3$ denote the degree of parallelism. To ensure that the parallel graph has the same input/output behavior as the original pipeline `map(f) >> map(g)`, we use round-robin distribution and collection, which implicitly preserves the ordering of items without requiring explicit sequence numbers.

We use the following notation: `read(ch)` blocks until an item is available on channel `ch` and returns it; `emit(item, ch)` sends an item on channel `ch`. All channels are FIFO.

split node. The `split` node distributes input items to the three `map(f)` instances in round-robin fashion. Let $out[0]$, $out[1]$, $out[2]$ be the three output channels.

```
// split: 1 input channel (in), 3 output channels (out[0..2])
lane = 0;
while (true) {
    A a = read(in);
    emit(a, out[lane]);
    lane = (lane + 1) % 3;
```

```
}
```

Item i (0-indexed) is sent to lane i mod 3. Thus lane 0 receives items $0, 3, 6, \ldots$; lane 1 receives items $1, 4, 7, \ldots$; and lane 2 receives items $2, 5, 8, \ldots$

map(f) instance i ($i \in \{0, 1, 2\}$). Each `map(f)` instance reads items from its input channel (from `split`), applies f, and sends the result to `map(g)`$_i$ on the lane-local output channel.

```
// map(f) instance i: input from split, output to map(g)[i]
while (true) {
    A a = read(in);
    B b = f(a);
    emit(b, out);
}
```

map(g) instance i ($i \in \{0, 1, 2\}$). Each `map(g)` instance reads intermediate results from `map(f)`$_i$, applies g, and sends the result to the `merge` node.

```
// map(g) instance i: input from map(f)[i], output to merge
while (true) {
    B b = read(in);
    C c = g(b);
    emit(c, out);
}
```

merge node. The `merge` node collects outputs from the three `map(g)` instances and emits them in the original input order. Since `split` distributes items round-robin, the order can be reconstructed by reading from the three input channels in the same round-robin fashion.

```
// merge: 3 input channels (in[0..2]), 1 output channel (out)
lane = 0;
while (true) {
    C c = read(in[lane]);
    emit(c, out);
    lane = (lane + 1) % 3;
}
```

Correctness. The input sequence $a_0, a_1, a_2, a_3, a_4, a_5, \ldots$ is distributed by `split` so that lane i receives items $a_i, a_{i+3}, a_{i+6}, \ldots$ Each `map(f)` and `map(g)` instance preserves the FIFO order. After applying f and g on each lane, lane i outputs $g(f(a_i)), g(f(a_{i+3})), g(f(a_{i+6})), \ldots$ The `merge` reads round-robin (lane 0, 1, 2, 0, 1, 2, ...), producing:

$$g(f(a_0)),\ g(f(a_1)),\ g(f(a_2)),\ g(f(a_3)),\ g(f(a_4)),\ g(f(a_5)),\ \ldots$$

This is exactly the output of the original pipeline `map(f) >> map(g)`.

Noteworthy changes to the graph.

1. **Simplification of cross-connections.** The graph in Figure 3 shows all nine connections between the `map(f)` and `map(g)` layers (every `map(f)` instance is connected to every `map(g)` instance). However, with round-robin distribution, each $\texttt{map(f)}_i$ only needs to communicate with $\texttt{map(g)}_i$ (lane-local routing). The six cross-lane connections are unnecessary and should be removed. If they were used and each `map(f)` instance broadcast its output to all three `map(g)` instances, items would be processed multiple times by different `map(g)` instances, producing incorrect (duplicated) output. Alternatively, if the cross-connections are retained, a routing mechanism must ensure each item reaches exactly one `map(g)` instance.

2. **All nodes are deterministic Kahn processes.** A notable advantage of the round-robin split/merge approach is that all nodes (including `merge`) are deterministic Kahn processes. The `merge` does not require a non-deterministic `select` operation—it simply reads from channels in a fixed round-robin order. This would not be possible if the cross-connections were used with arbitrary routing, since `map(g)` instances would then need to read from multiple input channels non-deterministically.

3. **Finite stream handling.** If the input stream has n items where n is not a multiple of 3, the lanes will have unequal numbers of items ($\lceil n/3 \rceil$ vs. $\lfloor n/3 \rfloor$). The round-robin merge will correctly output all n items before blocking on the next read from a shorter lane. This blocking is the standard Kahn-model behavior for the end of a finite stream and does not affect correctness.

1.5 Disorder and Watermarks [40 points]

Consider an IoT application that requires collecting data from a large and physically distributed collection of smart sensors. All sensor streams are sent to a centralized location ("server") for processing. Because of the physical distribution, the data can arrive disordered. The server sees a single input stream of data items of the form $(sensorId, val, ts) : \mathbb{N} \times \mathbb{R} \times \mathbb{N}$, where $sensorId : \mathbb{N}$ is a unique identifier for a sensor, $val : \mathbb{R}$ is the sensor measurement, and $ts : \mathbb{N}$ is a timestamp that measures time in msec (milliseconds). The data items in the input stream may be *disordered* with respect to the timestamp. So, we *cannot* expect in general that the data will appear in non-decreasing timestamp order.

We define the *watermark* of the input stream to be the maximum timestamp seen so far[1]. We make a *bounded lateness* assumption, namely that each data item should be at most $T \geq 0$ time units behind the watermark. That is, if $(sensorId, val, ts)$ is the current data item and t is the current watermark, then it should hold that $t - ts \leq T$.

We want to compute the *tumbling window average* of the measurement values. Suppose that $W > 0$ is the length of the window. For each time interval $[0, W), [W, 2W), [2W, 3W), \ldots$ we want to output the average value of all measurements that belong to the interval according to their timestamp. The output should be of the form $(avg, wndStartTs, wndEndTs) : \mathbb{R} \times \mathbb{N} \times \mathbb{N}$, where $avg : \mathbb{R}$ is the average measurement of the window, $\mathbb{N}$ is the start time of the window, and $\mathbb{N}$ is the end time of the window. E.g., if y_0 is the average of the window $[0, W)$, then the output should be $(y_0, 0, W - 1)$.

$$\xrightarrow{\mathbb{N} \times \mathbb{R} \times \mathbb{N}} \boxed{\texttt{tumbling-window(W, val, average)}} \xrightarrow{\mathbb{R} \times \mathbb{N} \times \mathbb{N}}$$

You should emit the output for a window as early as possible, i.e., as soon as it becomes apparent that no other measurement of the window will be seen in the future (recall the bounded lateness assumption).

Provide detailed streaming pseudocode for the most efficient implementation of this tumbling window operator that you can think of. Discuss the memory and time-per-item complexity of your implementation.

Solution:

Approach. We maintain a watermark (maximum timestamp seen so far) and accumulate per-window statistics (sum and count of measurement values). When the watermark advances far enough that no more items can arrive for a given window (under the bounded lateness assumption), we close that window and emit its average.

Window assignment. An item with timestamp ts belongs to window $k = \lfloor ts/W \rfloor$, which covers the time interval $[kW, (k+1)W)$.

Closing condition. Under the bounded lateness assumption, any future item has timestamp $ts \geq \text{watermark} - T$. This follows because the watermark is non-decreasing: after the current watermark reaches w, any subsequently arriving item with timestamp ts' satisfies $w' - ts' \leq T$ where

[1] Similar notions of watermarks are considered in many data stream processing systems. For example, see `https://learn.microsoft.com/en-us/azure/stream-analytics/stream-analytics-time-handling` and `https://www.databricks.com/blog/feature-deep-dive-watermarking-apache-spark-structured-streaming`.

$w' \geq w$ is the watermark at the time of that item's arrival. If $ts' \geq w'$, then $ts' \geq w' \geq w \geq w - T$. If $ts' < w'$, then $ts' \geq w' - T \geq w - T$. In both cases, $ts' \geq w - T$.

Window k covers timestamps up to $(k+1)W - 1$. Therefore, window k can be safely closed when no future item can have a timestamp in $[kW, (k+1)W)$, which occurs when:

$$\text{watermark} - T \geq (k+1)W \quad \Longleftrightarrow \quad \text{watermark} \geq (k+1)W + T$$

At this point, all future items will have timestamps $\geq (k+1)W$, placing them in window $k+1$ or later.

Pseudocode.

```
class TumblingWindowAvg
    implements Operator<(Nat, Real, Nat), (Real, Nat, Nat)> {

  final int W; // window size (msec)
  final int T; // max lateness (msec)

  // State
  int watermark = -1;
  int nextToClose = 0; // smallest window id not yet closed
  HashMap<Integer, Double>  sumMap   = new HashMap<>();
  HashMap<Integer, Integer> countMap = new HashMap<>();

  void start(Sink sink) { /* no output on start */ }

  void next((int sensorId, double val, int ts), Sink sink) {
    // 1. Update watermark
    watermark = max(watermark, ts);

    // 2. Assign item to its tumbling window
    int wid = ts / W; // floor division = window id

    // 3. Update the window's running sum and count
    if (!sumMap.containsKey(wid)) {
      sumMap.put(wid, 0.0);
      countMap.put(wid, 0);
    }
    sumMap.put(wid, sumMap.get(wid) + val);
    countMap.put(wid, countMap.get(wid) + 1);

    // 4. Emit all closeable windows in chronological order
    while ((nextToClose + 1) * W + T <= watermark) {
      if (sumMap.containsKey(nextToClose)) {
        double avg   = sumMap.get(nextToClose)
                     / countMap.get(nextToClose);
        int wndStart = nextToClose * W;
        int wndEnd   = (nextToClose + 1) * W - 1;
        sink.next((avg, wndStart, wndEnd));
```

```
        sumMap.remove(nextToClose);
        countMap.remove(nextToClose);
      }
      // Skip empty windows (no items fell in this interval)
      nextToClose++;
    }
  }
}
```

Detailed explanation.

1. **Watermark update (line 18):** On each incoming item, we update the watermark to be the maximum of the current watermark and the item's timestamp. The watermark is monotonically non-decreasing.

2. **Window assignment (line 21):** The window ID is computed as $\lfloor ts/W \rfloor$, placing the item in the tumbling window $[kW, (k+1)W)$ where $k = \lfloor ts/W \rfloor$.

3. **Accumulator update (lines 23–28):** We maintain a running sum and count for each active window. This allows computing the average when the window closes, *without storing individual data items*. Only two numbers per window are needed.

4. **Window closing (lines 31–40):** We maintain a variable `nextToClose` that tracks the smallest window ID that has not yet been closed. After each watermark update, we check in a while loop whether pending windows can be closed. A window with ID k is closeable when $(k+1) \cdot W + T \leq$ watermark.

   If the window accumulated items, we compute and emit the average. Windows with no items are silently skipped (no average to emit). The `nextToClose` pointer advances monotonically, ensuring windows are emitted in chronological order and each window is processed at most once.

5. **Earliest possible emission:** The output for each window is emitted immediately when the closing condition is first satisfied by a watermark advance. This is the earliest possible moment: before this point, the bounded lateness assumption still allows items to arrive for the window, so we cannot emit sooner without risking an incorrect average.

6. **No late items reach closed windows:** Under the bounded lateness assumption, once we close window k (when watermark $\geq (k+1)W+T$), any future item satisfies $ts \geq$ watermark$-T \geq (k+1)W$. Such items belong to window $k+1$ or later, so they never need to update a closed window. The invariant $wid \geq nextToClose$ always holds.

Complexity analysis.

- **Time per item: $O(1)$ amortized.** The watermark update, window assignment, and accumulator update are each $O(1)$ (hash map operations with expected constant-time lookup). The while loop for closing windows runs at most once per window over the entire execution of the stream. Since each window is closed exactly once, the total closing work across all n items is bounded by the total number of windows (at most n windows, but typically far fewer). By an amortized argument, the cost is $O(1)$ per item.

- **Memory: $O(T/W+1)$.** At any point, the active (not yet closed) windows are those covering timestamps in the range [watermark $- T$, watermark]. The number of distinct tumbling windows that intersect this range is at most:

$$\left\lfloor \frac{T}{W} \right\rfloor + 2 \;=\; O\left(\frac{T}{W} + 1\right)$$

  Each window stores only a sum and a count ($O(1)$ space per window), so the total memory for window accumulators is $O(T/W + 1)$. Importantly, we do *not* store individual data items—only aggregate statistics. The `sensorId` field is not used for the global average computation.

  When $T < W$ (lateness is less than one window), at most 2 windows are active at any time, giving $O(1)$ memory. When $T \gg W$ (high lateness relative to window size), many windows may be simultaneously active, reflecting the inherent cost of handling disorder.

2 Denotational Semantics of Kahn Process Networks [40 points]

Lemma 2. Let A and B be ω-CPOs. Then, $A \times B$ is an ω-CPO.

Proof. We equip $A \times B$ with the componentwise partial order: $(a_1, b_1) \sqsubseteq (a_2, b_2)$ iff $a_1 \sqsubseteq_A a_2$ and $b_1 \sqsubseteq_B b_2$. We verify the two requirements of an ω-CPO.

Least element. Since A and B are ω-CPOs, they have least elements $\bot_A$ and $\bot_B$ respectively. We claim $(\bot_A, \bot_B)$ is the least element of $A \times B$. For any $(a, b) \in A \times B$, we have $\bot_A \sqsubseteq_A a$ and $\bot_B \sqsubseteq_B b$, so $(\bot_A, \bot_B) \sqsubseteq (a, b)$.

Least upper bounds of ω-chains. Let $(a_0, b_0) \sqsubseteq (a_1, b_1) \sqsubseteq (a_2, b_2) \sqsubseteq \cdots$ be an ω-chain in $A \times B$. By the definition of the componentwise order, this gives rise to two ω-chains:

$$a_0 \sqsubseteq_A a_1 \sqsubseteq_A a_2 \sqsubseteq_A \cdots \quad \text{in } A, \qquad b_0 \sqsubseteq_B b_1 \sqsubseteq_B b_2 \sqsubseteq_B \cdots \quad \text{in } B.$$

Since A is an ω-CPO, the chain $(a_n)_{n \geq 0}$ has a least upper bound $a = \bigsqcup_n a_n$ in A. Since B is an ω-CPO, the chain $(b_n)_{n \geq 0}$ has a least upper bound $b = \bigsqcup_n b_n$ in B.

We claim that $(a, b) = \bigsqcup_n (a_n, b_n)$ in $A \times B$.

Upper bound: For all $n \geq 0$, we have $a_n \sqsubseteq_A a$ and $b_n \sqsubseteq_B b$, so $(a_n, b_n) \sqsubseteq (a, b)$.

Least upper bound: Let (a', b') be any upper bound of the chain, i.e., $(a_n, b_n) \sqsubseteq (a', b')$ for all n. Then $a_n \sqsubseteq_A a'$ for all n, so a' is an upper bound of the chain $(a_n)_{n \geq 0}$ in A. Since $a = \bigsqcup_n a_n$ is the *least* upper bound, we have $a \sqsubseteq_A a'$. By the same argument, $b \sqsubseteq_B b'$. Therefore $(a, b) \sqsubseteq (a', b')$.

This shows that every ω-chain in $A \times B$ has a least upper bound. Hence $A \times B$ is an ω-CPO. $\square$

Lemma 3. Let A, B and C be ω-CPOs. Suppose that $f : A \to B$ and $g : B \to C$ are ω-continuous. Then, the serial composition $f \gg g : A \to C$ is ω-continuous. Recall from the lectures that $(f \gg g)(a) = g(f(a))$ for every $a \in A$.

Proof. We need to show that $f \gg g$ is ω-continuous, i.e., (1) monotone and (2) preserving least upper bounds of ω-chains.

Monotonicity. Let $a_1 \sqsubseteq_A a_2$. Since f is ω-continuous, it is monotone, so $f(a_1) \sqsubseteq_B f(a_2)$. Since g is ω-continuous, it is monotone, so $g(f(a_1)) \sqsubseteq_C g(f(a_2))$. Therefore $(f \gg g)(a_1) \sqsubseteq_C (f \gg g)(a_2)$.

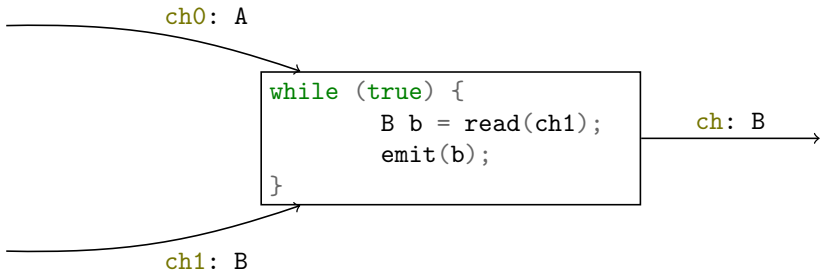
Preservation of ω-chain suprema. Let $a_0 \sqsubseteq a_1 \sqsubseteq a_2 \sqsubseteq \cdots$ be an ω-chain in A. We compute:

$$\begin{aligned}
(f \gg g)\left(\bigsqcup_n a_n\right) &= g\left(f\left(\bigsqcup_n a_n\right)\right) \\
&= g\left(\bigsqcup_n f(a_n)\right) && (f \text{ is } \omega\text{-continuous}) \\
&= \bigsqcup_n g(f(a_n)) && (g \text{ is } \omega\text{-continuous; see below}) \\
&= \bigsqcup_n (f \gg g)(a_n)
\end{aligned}$$

In the third step, we applied the ω-continuity of g to the sequence $f(a_0), f(a_1), f(a_2), \ldots$ in B. This is indeed an ω-chain because f is monotone: $a_i \sqsubseteq_A a_{i+1}$ implies $f(a_i) \sqsubseteq_B f(a_{i+1})$ for all $i \geq 0$. Therefore the ω-continuity of g applies, yielding $g(\bigsqcup_n f(a_n)) = \bigsqcup_n g(f(a_n))$.

Hence $f \gg g$ is ω-continuous. $\square$

We will consider now the ***semantics of feedback*** using an interesting example. Consider the Kahn process shown below:



The (cumulative) input/output behavior of this process can be described by an ω-continuous function $f : A^\infty \times B^\infty \to B^\infty$. Describe the function f and the function $\mathsf{loop}(f) : A^\infty \to B^\infty$ (loop is feedback composition, as defined in the lectures).

Solution: **Fill me in.**

3 ToyDSL [100 points]

Download the Java project form Canvas. Carefully read all the provided code in order to understand the programming model that we consider. Implement the ToyDSL domain-specific language using the included instructions. Your implementation should pass all included unit tests. If there is anything that you would like to clarify, please do so below.

Solution

Fill me in.

References

[1] Gilles Kahn. The semantics of a simple language for parallel programming. *Information Processing*, 74:471–475, 1974.

[2] Gilles Kahn and David B. MacQueen. Coroutines and networks of parallel processes. *Information Processing*, 77:993–998, 1977.

[3] Edward A. Lee. *Concurrent Models of Computation: An Actor-Oriented Approach*. 2011.

[4] David B. MacQueen. Kahn networks at the dawn of functional programming. In Yves Bertot, Gérard Huet, Jean-Jacques Lévy, and Gordon Plotkin, editors, *From Semantics to Computer Science: Essays in Honour of Gilles Kahn*, pages 95–137. Cambridge University Press, 2009.